LAPORAN TUGAS BESAR

IF4031 Arsitektur Aplikasi Terdistribusi

Milestone 1

Sistem Terdistribusi Dasar

Platform Koordinasi Kebencanaan BNPB



DikejarDeadline

Refki Alfarizi	13523002
M Hazim R Prajoda	13523009
Adhimas Aryo Bimo	13523052

PROGRAM STUDI TEKNIK INFORMATIKA

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

DAFTAR ISI

1 DESKRIPSI UMUM SISTEM **1**
1.1 Latar Belakang . . . 1
1.2 Tujuan . . . 1
1.3 Ruang Lingkup . . . 1
1.4 Aktor dan Batas Sistem . . . 2
1.5 Istilah dan Data Kanonik . . . 2

2 ARSITEKTUR SISTEM **3**
2.1 Diagram Arsitektur . . . 3
2.2 Komponen dan Tanggung Jawab . . . 4
2.3 Alasan Pembagian Service . . . 4
2.4 Model Kelas Data . . . 5
2.5 Protokol dan Alur Komunikasi . . . 5
2.6 Kepemilikan Storage dan Container . . . 6
2.7 Alur Permintaan dan Kegagalan . . . 6
2.7.1 Batas Jaringan . . . 6

3 TEKNOLOGI DAN ASUMSI **7**
3.1 Teknologi Terpilih . . . 7
3.2 Alternatif Teknologi . . . 7
3.3 Asumsi dan Batas Implementasi . . . 7
3.4 Konfigurasi dan Observabilitas . . . 8
3.5 Kontrak Mock dan Data Awal . . . 8
3.6 Menjalankan dan Memverifikasi . . . 9
3.6.1 Urutan Menjalankan Demo . . . 9
3.6.2 Lingkungan Evidence . . . 9

4 PROBLEM 1 **10**
4.1 Masalah dan Kriteria . . . 10
4.2 Solusi yang Diimplementasikan . . . 10
4.2.1 Ringkasan Pemetaan Sumber . . . 11
4.3 Evolusi Skema dan Trade-off . . . 11
4.4 Batasan yang Disadari . . . 11
4.5 Bukti Penyelesaian . . . 11
4.5.1 Bukti Integrasi . . . 12
4.5.2 Pemetaan Bukti ke Kriteria . . . 13
4.5.3 Temuan pada Volume Lebih Besar . . . 13

5 PROBLEM 2 **14**
5.1 Isolasi request dan polling . . . 14

5.2 Outage dan pemulihan . . . 14
5.3 Metode dan hasil load test . . . 16
5.4 Batasan . . . 19

6 PROBLEM 3 **20**
6.1 Credential dan hak akses . . . 20
6.2 Expiry dan refresh . . . 20
6.3 Pengujian credential dan hak akses . . . 21
6.3.1 Credential hanya berlaku pada sumbernya . . . 21
6.3.2 Media menerima ringkasan dan tidak dapat meminta data raw . . . 22
6.4 Pengujian expiry dan refresh . . . 23
6.5 Menelusuri request dengan correlation ID . . . 26
6.6 Batasan keamanan dan waktu . . . 27

7 PROBLEM 4 **28**
7.1 Container dan ownership . . . 28
7.2 Detail Implementasi Canonical Store . . . 28
7.3 Struktur Storage dan Alternatif . . . 29
7.4 Pengujian penggantian satu container . . . 29
7.5 Pengujian ownership dan flexible storage . . . 31
7.6 Verifikasi Komponen Storage . . . 32
7.7 Batasan . . . 32

8 PROBLEM 5 **33**
8.1 Producer, stream, dan consumer . . . 33
8.2 Semantik pengiriman dan idempotensi . . . 34
8.3 Pengujian fan-out dan recovery . . . 35
8.3.1 Mencatat posisi dua durable consumer . . . 35
8.3.2 Menghentikan satu consumer dan membaca backlog . . . 36
8.3.3 Menambahkan consumer tanpa mengubah producer . . . 39
8.4 Batasan . . . 41

9 KESIMPULAN **42**

10 PEMBAGIAN TUGAS **43**

DAFTAR REFERENSI **44**

DESKRIPSI UMUM SISTEM

1.1. Latar Belakang

BMKG dan PVMBG mengelola data dari domain serta skema yang berbeda, sedangkan BNPB perlu menyajikannya secara konsisten kepada beberapa jenis client. Pada milestone 1, kelompok mengimplementasikan platform koordinasi terdistribusi yang mengambil data kedua sumber, menormalkannya ke model kanonik, menyimpan hasilnya, dan mendistribusikan event baru tanpa membuat producer bergantung langsung pada consumer.

1.2. Tujuan

Tujuan sistem adalah: (1) mengintegrasikan sumber seismik dan vulkanik tanpa mengubah kontrak mock; (2) tetap melayani data BMKG ketika PVMBG lambat atau tidak tersedia; (3) memisahkan kredensial sumber dan hak akses client; (4) memisahkan komponen agar dapat dijalankan dan dipulihkan secara independen; serta (5) mendistribusikan event baru melalui pub/sub. Fitur dan ambang pengujian dijelaskan per problem pada Bab 4–8.

1.3. Ruang Lingkup

milestone 1 dibatasi pada mock BMKG dan PVMBG, BNPB sebagai node koordinasi, serta dua tipe bahaya: seismik dan vulkanik. Client downstream adalah Media/Pers, Tim Lapangan, dan BNPB Pusat/Internal; seluruh akses client bersifat baca-saja. Pelaporan dari lapangan, kebakaran hutan, dan jenis bahaya lain berada di luar cakupan milestone 1 [1,2]. Notifier mencatat alert pada storage lokal sebagai simulasi, bukan mengirim SMS sungguhan. Demo console adalah alat operator, bukan Dashboard Consumer dan bukan prasyarat menjalankan service.

1.4. Aktor dan Batas Sistem

Aktor/komponen	Tanggung jawab	Batas akses
Mock BMKG	Menyediakan event gempa dan peringatan tsunami sesuai kontrak simulasi.	Endpoint data memakai kredensial BMKG; port demo terikat ke loopback host.
Mock PVMBG	Menyediakan laporan gunung api, perubahan skema aditif, dan kontrol outage.	Endpoint data dan admin memakai kredensial PVMBG yang berbeda; port demo terikat ke loopback host.
BNPB Aggregator	Polling, validasi, korelasi, pemetaan ke HazardEvent, riwayat revisi, API internal, dan relay outbox.	Satu-satunya pemilik dan pengakses langsung Canonical Store.
Client-Facing API dan Auth/Token	Menyediakan akses client, menerbitkan/mengecek token, dan menegakkan scope di server.	Tidak membuka koneksi langsung ke Canonical Store.
Consumer downstream	Memproses event secara independen, misalnya pembaruan dashboard dan notifikasi.	Berlangganan stream; tidak dipanggil langsung oleh producer.

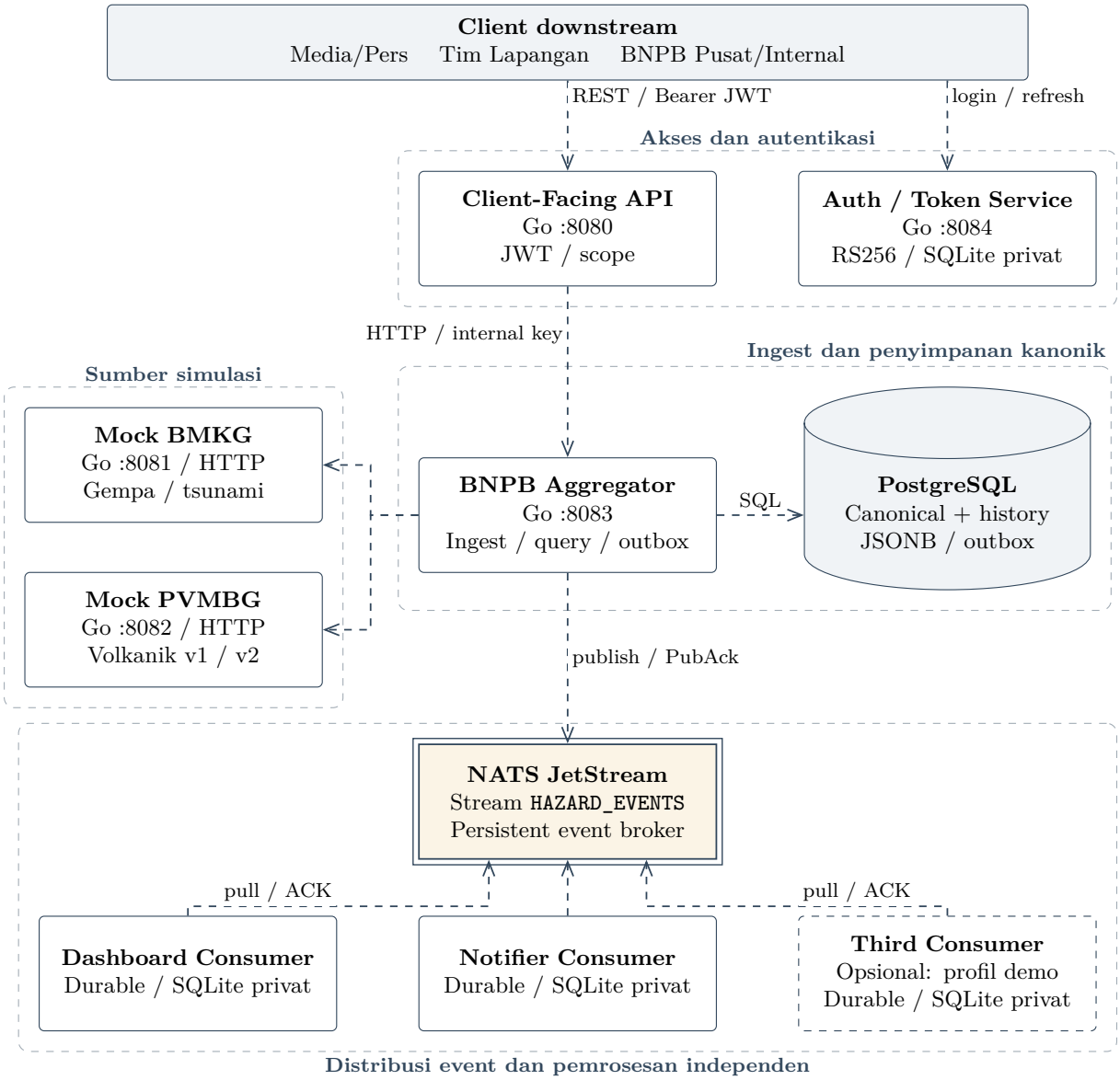
1.5. Istilah dan Data Kanonik

`SeismicEvent` dan `TsunamiWarning` berasal dari BMKG; warning diambil lewat endpoint terpisah dan dikorelasikan memakai `related_event_id`. `VolcanicReport` berasal dari PVMBG. Aggregator memetakan keduanya ke `HazardEvent`, yang memisahkan field ringkasan dari data mentah. Atribut spesifik sumber atau field baru seperti `confidence_level` ditempatkan dalam `attributes`; aturan lengkapnya ada di Bab 4.

ARSITEKTUR SISTEM

2.1. Diagram Arsitektur

Gambar 2.1 memperlihatkan arsitektur sistem dengan pengelompokan tanggung jawab, bentuk storage yang berbeda, serta dependency antarservice. Ini adalah diagram arsitektur logis, bukan notasi UML murni atau diagram deployment jaringan. Panah putus-putus menunjuk dari pemakai layanan ke penyedianya; jalur event dijelaskan pada Bab 8.



Gambar 2.1: Arsitektur logis milestone 1. Kotak menunjukkan service; silinder adalah Canonical Store dan kotak ganda adalah broker. Third Consumer bersifat opsional. Bingkai mengelompokkan tanggung jawab, bukan batas Docker network. Panah consumer menuju broker menunjukkan dependency layanan, bukan arah delivery event.

2.2. Komponen dan Tanggung Jawab

- **Mock BMKG/PVMBG**: service pasif yang hanya merespons polling. Endpoint data mengikuti kontrak tugas; endpoint admin tambahan mendukung simulasi delay dan outage.
- **Aggregator**: menjalankan poller BMKG dan PVMBG secara terisolasi, memvalidasi payload, menggabungkan tsunami warning dengan event gempa, memetakan data, menyimpan canonical record, serta mengekspos API internal untuk query Client-Facing API.
- **Client-Facing API**: memeriksa scope, meminta data melalui API Aggregator, lalu menyaring field di server. Komponen ini tidak membaca PostgreSQL secara langsung.
- **Auth/Token Service**: menjaga identitas client yang berbeda, JWT RS256 berumur pendek dan state refresh pada SQLite privat. Client API memvalidasi JWT secara lokal dengan public key; permintaan baca tidak membutuhkan panggilan Auth.
- **NATS JetStream dan consumer**: Aggregator menerbitkan event; consumer dashboard dan notifikasi memiliki durable consumer masing-masing pada stream yang sama.

Pemisahan ini menjaga batas kepemilikan storage dan memungkinkan satu container dihentikan atau dibangun ulang tanpa menggabungkan seluruh sistem menjadi satu unit deploy [1].

2.3. Alasan Pembagian Service

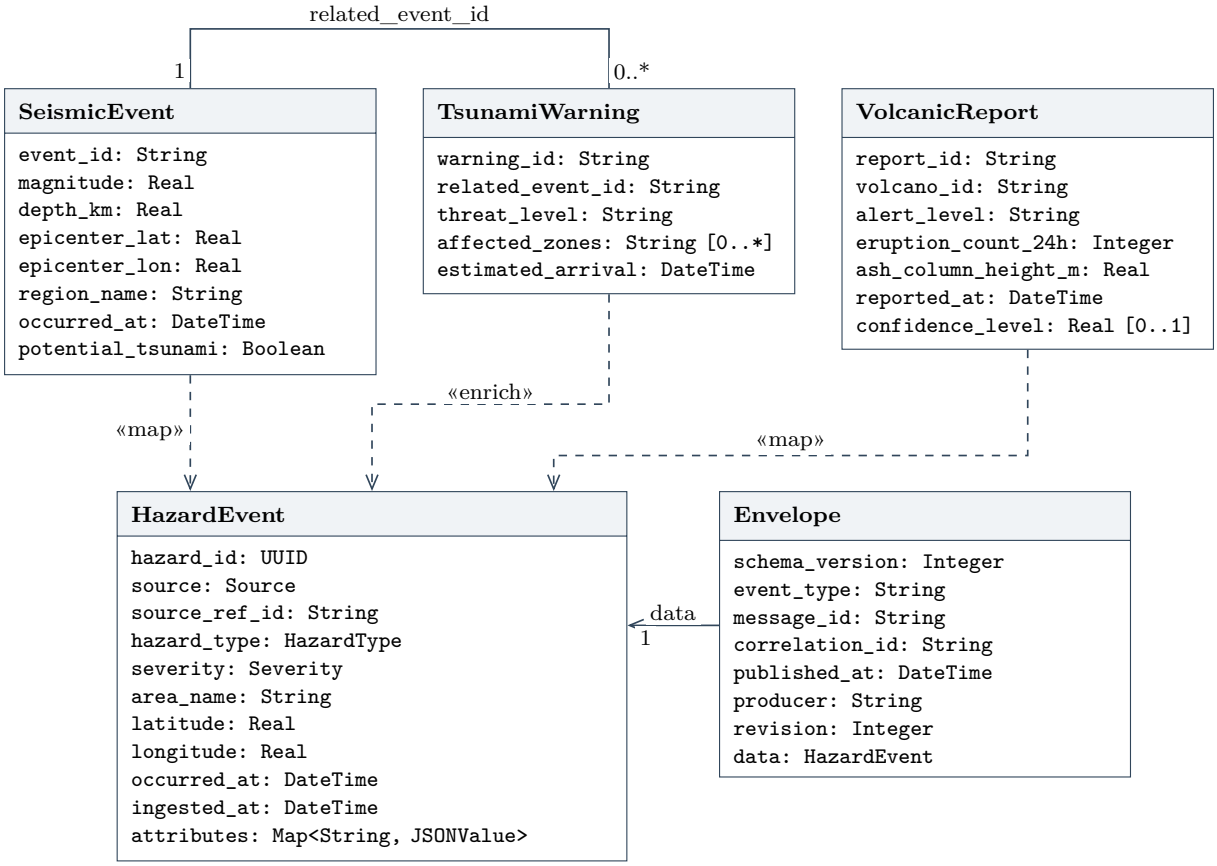
Aggregator menyatukan polling, mapping, dan relay karena semuanya menggunakan Canonical Store yang sama. Loop dan pool koneksi masing-masing pekerjaan dipisah, tetapi seluruhnya tetap berhenti jika container Aggregator mati. Memisahkan ingest per instansi dapat memberi isolasi deployment tambahan, dengan biaya antarmuka dan koordinasi baru.

Client API dipisahkan dari Auth agar proses pembacaan cukup memegang public key, sedangkan private key penandatanganan tetap pada Auth. Gangguan Auth tidak membatalkan access token yang masih valid karena signature diperiksa secara lokal; login dan refresh tetap membutuhkan Auth. Pemisahan ini menambah panggilan HTTP dari API ke Aggregator, tetapi menjaga ownership database dan memisahkan perubahan akses client dari ingest.

Dashboard dan Notifier mempunyai container, durable, serta storage terpisah. Jika keduanya memakai satu durable, pesan dibagi sebagai pekerjaan, bukan disalin untuk setiap aplikasi. Durable terpisah memenuhi fan-out dan memungkinkan masing-masing mengejar backlog. Third Consumer memakai binary Notifier dengan durable dan volume sendiri, melalui profil `demo`; producer tidak perlu mengetahui consumer tambahan.

2.4. Model Kelas Data

Gambar 2.2 menggunakan UML Class Diagram untuk kontrak JSON, bukan struktur tabel database. Association gempa–warning mengikuti identifier sumber. Dependency «map» dan «enrich» adalah label pemetaan lokal, bukan inheritance. `DateTime` diserialisasikan sebagai string timestamp; `JSONValue` dapat berupa object, array, angka, string, boolean, atau null.



Gambar 2.2: Kontrak sumber, entity kanonik, dan envelope event. Satu warning merujuk satu gempa; satu gempa dapat mempunyai banyak warning. `confidence_level` bersifat opsional dan muncul pada PVMBG v2. `Envelope.data` berisi satu snapshot HazardEvent, bukan referensi ke latest row.

Enum kanonik: `Source` = BMKG/PVMBG; `HazardType` = SEISMIC/VOLCANIC; `Severity` = NORMAL/WASPADA/SIAGA/AWAS. Field tambahan sumber dipertahankan dalam `attributes`; metadata revisi dan delivery tetap berada di envelope. Warning yang belum memiliki gempa terkait dapat berada dalam snapshot sumber, tetapi tidak menjadi hazard mandiri.

2.5. Protokol dan Alur Komunikasi

1. Poller BMKG mengambil `/seismic-events` dan `/tsunami-warnings` secara concurrent. Poller PVMBG mengambil `/volcanic-reports` secara independen. Keduanya menggunakan full snapshot, bukan cursor `since`.

2. Aggregator memvalidasi dan memetakan data, lalu menyimpan latest hazard, revisi, dan outbox dalam satu transaksi. Relay menandai pesan terpublikasi setelah PubAck JetStream.
3. Client API memvalidasi JWT secara lokal dan membaca API internal Aggregator. Media hanya menerima ringkasan tanpa koordinat presisi, identifier sumber, atau attributes.
4. Consumer membaca stream secara independen dan ACK setelah pemrosesan berhasil. Detail mapping dan delivery ada pada Bab 4 dan 8.

2.6. Kepemilikan Storage dan Container

PostgreSQL adalah Canonical Store milik Aggregator. Data terbaru berada pada `hazard_events`; riwayatnya berada pada `hazard_versions`. Tabel `source_state` menyimpan freshness, dan `outbox` menyimpan envelope delivery. Field kanonik bertipe; attributes memakai JSONB. Auth memiliki SQLite untuk state refresh; setiap consumer memiliki SQLite untuk inbox dan side effect. Service lain tidak membuka koneksi ke Canonical Store. Sembilan service utama berjalan dalam container terpisah melalui Compose.

2.7. Alur Permintaan dan Kegagalan

Polling berjalan di background, bukan saat request client. Delay atau outage PVMBG tidak menahan BMKG maupun API baca. Data terakhir dapat ditandai stale/unavailable; full snapshot setelah pemulihan memperbarui freshness tanpa restart. Kegagalan broker menyisakan pending outbox; kegagalan consumer tidak menahan publish. Durable consumer dapat menerima redelivery selama pesan masih dalam retensi.

2.7.1 Batas Jaringan

Compose memisahkan `source-net`, `api-net`, `store-net`, dan `event-net`. Dua jaringan terakhir bersifat internal dan PostgreSQL/NATS tidak memublikasikan port host. Port aplikasi demo terikat ke `127.0.0.1`; deployment lintas host memerlukan TLS.

TEKNOLOGI DAN ASUMSI

3.1. Teknologi Terpilih

Implementasi menggunakan Go dengan modul minimum Go 1.23. Driver PostgreSQL adalah `pgx/v5`, broker diakses melalui `nats.go`, dan validasi fixture kontrak memakai JSON Schema Draft 2020-12. Compose memilih PostgreSQL 16 dan NATS 2.10; tag tersebut belum mengunci image digest. Auth menggunakan library JWT dan SQLite, sedangkan masing-masing consumer memiliki SQLite privat. Pilihan ini memisahkan canonical data dari state autentikasi dan idempotensi consumer.

Go dipakai untuk HTTP, parsing JSON, goroutine, dan pembatalan melalui `context`. PostgreSQL menyediakan transaksi untuk perubahan hazard, riwayat, dan outbox; JSONB menjaga atribut dinamis tanpa mengubah kolom kanonik [3,4,5]. JetStream menyediakan persistent stream dan durable consumer, bukan jaminan exactly-once menyeluruh [6,7]. Docker Compose mengelola service terpisah pada satu host [9].

3.2. Alternatif Teknologi

- **PostgreSQL + JSONB**: kolom bertipe menjaga identitas dan constraint, sedangkan atribut aditif tetap berupa dokumen. Dibanding seluruh atribut menjadi kolom, tambahan field tidak membutuhkan migrasi; dibanding document store terpisah, transaksi latest/history/outbox tetap berada pada satu store.
- **JetStream**: menambahkan persistence, replay, dan posisi durable pada NATS. NATS Core tidak menyimpan backlog untuk consumer offline. Kafka atau RabbitMQ juga dapat mendukung distribusi event, tetapi menambah pilihan konfigurasi yang belum diperlukan untuk demonstrasi satu host ini.
- **SQLite privat**: sesi refresh dan inbox tetap tersimpan saat restart, tanpa server database tambahan. Penyimpanan memory lebih sederhana tetapi kehilangan state; beberapa replica yang harus berbagi state memerlukan rancangan database dan koordinasi berbeda.
- **Docker Compose**: cukup untuk container, network, dan volume pada satu host. Kubernetes belum diperlukan untuk cakupan demo; konfigurasi Compose ini juga tidak memberikan failover lintas mesin.

3.3. Asumsi dan Batas Implementasi

1. Mock menggunakan fixture sintetis, bukan data produksi BMKG/PVMBG. Nama dan koordinat gunung pada referensi Aggregator diberi label simulasi Tamriel; tidak mewakili lokasi gunung sebenarnya.
2. Dua loop polling independen berjalan segera saat startup, lalu mengikuti interval default 5 detik. Satu sumber hanya memiliki satu siklus aktif; tick saat sibuk dilewati. BMKG mengambil seismic dan warning secara concurrent.
3. Baseline ingest menggunakan full snapshot, bukan cursor `since`. Endpoint mock

tetap mendukung `since`; full snapshot dipilih agar warning terlambat dan perubahan field pada ID lama tidak terlewat. Biaya baca bertambah bersama riwayat sumber.
4. Hanya perubahan aditif yang didukung otomatis. Field wajib hilang, tipe salah, enum tak dikenal, duplicate JSON key, atau volcano ID tak dikenal ditolak per record; record valid tetap diproses.
5. Baseline memakai satu Aggregator, satu database, dan satu node broker. Replikasi, failover lintas host, TLS produksi, dan exactly-once global berada di luar klaim implementasi ini.

3.4. Konfigurasi dan Observabilitas

Repository menyediakan `.env.example` tanpa secret aktif; credential lokal tidak masuk Git. Setiap service menerima subset konfigurasi miliknya. API key BMKG, token PVMBG, internal key Aggregator, dan token NATS dipisahkan.

Parameter Aggregator	Default implementasi
Interval poll / stale threshold	5 s / 15 s
Timeout HTTP BMKG / PVMBG	1 s / 4 s
Batas respons sumber / envelope event	1 MiB / 64 KiB
Pool baca / tulis BMKG / tulis PVMBG / relay	10 / 2 / 2 / 2 koneksi
Deadline query baca / transaksi tulis	200 ms / 2 s
Admission API internal	128 request data aktif
Relay poll / batch / pending maksimum	500 ms / 20 pesan / 10.000 pesan
Timeout publish / rentang retry	2 s / 1–30 s dengan jitter

Nilai tersebut merupakan konfigurasi awal, bukan hasil benchmark. `/health` menunjukkan liveness; `/ready` menilai dependency sesuai peran service. Kegagalan sumber atau broker tidak dengan sendirinya membuat API baca Aggregator unready selama database masih dapat dibaca. Log terstruktur menyertakan operasi, correlation ID, durasi, dan status tanpa payload mentah atau credential. Saat shutdown, context dibatalkan dan worker poller serta relay ditunggu sebelum resource ditutup.

3.5. Kontrak Mock dan Data Awal

BMKG melayani `/seismic-events` dan `/tsunami-warnings` dengan header `X-BMKG-Key`; delay default 100 ms. PVMBG melayani `/volcanic-reports` dengan Bearer token dan delay acak 500–3.000 ms. Endpoint admin `/admin/schema-version`, `/admin/outage`, serta tambahan `/admin/delay` menggunakan credential PVMBG yang sama. Credential diperiksa sebelum delay.

Kedua mock memulai dengan 20 record historis dan menambah record setiap 10 detik pada konfigurasi default. ID memuat run ID agar restart mock tidak menimpa riwayat lama. Toggle PVMBG v2 menambahkan `confidence_level` pada serialisasi seluruh laporan; outage memberi 503 untuk data, tetapi endpoint admin dan liveness tetap dapat diakses.

3.6. Menjalankan dan Memverifikasi

Definisi Compose di repository sudah mencakup sembilan service utama; consumer ketiga tersedia melalui profil demo. Setup mencakup secret berbeda dan key JWT sebelum menjalankan `docker compose up -build`. Pengujian deployment penuh dan tes komponen native memakai lingkungan berbeda.

Komponen diverifikasi dengan tes unit/HTTP, race detector, build, vet, smoke kedua mock, serta tes Store dan publisher terhadap PostgreSQL/NATS disposable. Pemeriksaan terbaru pada commit `19beead` lulus. Dependency native yang dipakai adalah PostgreSQL 18.6 dan NATS 2.10.24; karena Compose menargetkan PostgreSQL 16, hasil ini bukan pengganti verifikasi image Compose. Rincian mapping dan storage ada pada Bab 4 dan 7. Evidence stack Compose dan pengujian lintas layanan tersedia pada lingkungan berbeda [10]. Bukti tersebut dipisahkan dari tes native; keberadaan evidence tidak berarti seluruh acceptance telah diulang pada commit main atau setiap profil volume data.

3.6.1 Urutan Menjalankan Demo

Dari root repository, salin `.env.example` menjadi `.env`, isi credential yang berbeda untuk setiap domain dan client, serta tentukan `COMPOSE_PROJECT_NAME`. Parameter biasa dapat memakai default Compose. Buat pasangan RSA pada `.secrets/`, lalu muat ke volume private/public yang terpisah. Langkah lengkap berada di `scripts/README.md`; jangan mengganti key yang sudah dipakai tanpa mempertimbangkan token aktif.

```
python3 scripts/setup-jwt-keys.py
docker compose config -quiet
docker compose up -build -d -wait
```

Healthcheck menilai kesiapan startup, bukan seluruh acceptance. Login dilakukan ke Auth dengan identitas client yang sesuai sebelum mengakses API. Untuk menambahkan consumer ketiga tanpa membangun ulang producer:

```
docker compose -profile demo up -d -no-deps third-consumer
```

Pengujian integrasi dijalankan melalui `scripts/verify-integration.py` dengan opsi `-scenario all`. Script tersebut mengubah schema/outage, menghentikan service sementara, dan membangun ulang consumer; gunakan stack khusus eksperimen. Demo console dapat membantu operator menjalankan command, tetapi hanya tersedia di branch tooling demo dan tidak diperlukan untuk jalur CLI.

3.6.2 Lingkungan Evidence

Evidence mencatat trial 5 Oktober 2026 pada commit `071b32d`, yaitu branch tooling demo, bukan HEAD main `19beead` [10]. Lingkungan trial adalah WSL2 Linux, 12 CPU logical, Docker 27.0.3, Compose 2.28.1, PostgreSQL 16.14, dan NATS 2.10.29. Build image aplikasi memakai Go 1.23; pemeriksaan lokal menggunakan Go 1.26.4. Port host API sampai Auth memakai 8280–8284. Seed 20 record, interval event 10 detik, polling 5 detik, serta deadline write 2 detik tetap merupakan profil awal, bukan jaminan skalabilitas.

PROBLEM 1

ARSITEKTUR DAN INTEROPERABILITAS DATA

4.1. Masalah dan Kriteria

BMKG memisahkan gempa dan warning tsunami, sedangkan PVMBG memakai laporan gunung api tanpa koordinat langsung. Aggregator perlu menghasilkan model kanonik yang konsisten dan tetap memproses laporan ketika PVMBG menambah `confidence_level`, tanpa restart atau perubahan tabel [1]. Normalisasi tidak boleh membuang field tambahan atau menyamarkan record invalid sebagai data valid.

4.2. Solusi yang Diimplementasikan

Mapper membaca field sebagai `json.RawMessage`, memvalidasi field wajib, lalu mempertahankan nilai dinamis menggunakan `json.Number`. Output `HazardEvent` memiliki sebelas field: ID hazard, source, source reference, tipe, severity, area, latitude, longitude, waktu kejadian, waktu ingest, dan attributes. Metadata delivery ditempatkan pada envelope, bukan ditambahkan ke entity.

Untuk BMKG, `event_id` menjadi `source_ref_id`, region dan koordinat episentrum dipertahankan, dan tipe menjadi `SEISMIC`. Warning divalidasi secara mandiri sebelum dikelompokkan berdasarkan `related_event_id`. Warning rusak menandai siklus degraded tanpa menggugurkan gempa valid. Warning hanya terpasang pada gempa dengan `potential_tsunami=true`; jika ada beberapa warning, severity tertinggi dipilih dan seluruh warning valid disimpan terurut dalam attributes.

Tanpa warning, magnitude di bawah 5,0 menjadi NORMAL; 5,0 hingga kurang dari 6,5 menjadi WASPADA; dan mulai 6,5 menjadi SIAGA. Untuk PVMBG, `report_id` menjadi source reference, `reported_at` menjadi waktu kejadian, dan enum alert dipetakan secara eksplisit ke enum kanonik. Area dan koordinat diambil dari referensi sintetis Tamriel berdasarkan `volcano_id`; ID tak dikenal ditolak, bukan diberi koordinat nol.

Identitas permanen mengikuti pasangan `(source, source_ref_id)`. Data pertama memperoleh UUIDv4; perubahan isi mempertahankan UUID dan menghasilkan revisi baru. Full snapshot memungkinkan warning yang baru tersedia memengaruhi hazard lama pada poll berikutnya. Poll identik hanya memajukan freshness sumber dan tidak membuat event/outbox duplikat.

4.2.1 Ringkasan Pemetaan Sumber

Field kanonik	BMKG	PVMBG
`source_ref_id`	`event_id`	`report_id`
`hazard_type`	SEISMIC	VOLCANIC
Area dan koordinat	Region dan episentrum sumber	Lookup fixture berdasarkan ID gunung
Waktu kejadian	`occurred_at`	`reported_at`
Severity	Batas magnitude atau warning terkait	Alert level ke enum kanonik
Attributes	Magnitude, kedalaman, potensi tsunami, warning, extension	ID gunung, erupsi, tinggi abu, confidence jika tersedia, extension

ID hazard dibuat pada ingest pertama, sedangkan waktu ingest mencatat saat data diterima. Timestamp dinormalisasi ke UTC; bentuk serialisasi `Z` dan `+00:00` menunjukkan timezone yang sama.

4.3. Evolusi Skema dan Trade-off

Field tambahan yang tidak mempunyai padanan kanonik disimpan dalam `attributes`; collision dengan atribut hasil turunan dipertahankan dalam `source_extras`. Penambahan `confidence_level` tidak memerlukan migrasi tabel. Field wajib hilang, perubahan tipe, enum invalid, duplicate key, dan JSON invalid ditolak; siklus degraded tidak menaikkan waktu sukses terakhir.

Decode langsung ke struct lebih sederhana, tetapi dapat mengabaikan field tambahan. RawMessage dan validasi eksplisit dipilih agar evolusi aditif tidak menghilangkan data [4]. Full snapshot lebih mudah menjaga korelasi dan update record lama daripada cursor incremental, dengan trade-off biaya baca yang bertambah. Fleksibilitas JSONB tidak menggantikan validasi domain [5].

4.4. Batasan yang Disadari

Field baru disimpan tanpa otomatis menjadi field ringkasan atau mengubah severity. Rename atau penghapusan field wajib membutuhkan perubahan kontrak. Fixture Tamriel hanya untuk simulasi. Jalur warning terlambat tersedia, tetapi pengujian khusus dari poller sampai revisi PostgreSQL belum dijalankan sebagai satu skenario acceptance terpisah.

4.5. Bukti Penyelesaian

Tes mapper lulus untuk batas magnitude, prioritas dan korelasi warning, preservasi field dinamis, bentuk PVMBG v1/v2, serta penolakan enum/ID/duplicate key invalid. Tes poller lulus untuk warning rusak yang terkait maupun tidak terkait: gempa valid tetap diteruskan dan siklus ditandai degraded.

Smoke HTTP terbaru membuktikan seed 20 record, penolakan akses tanpa credential dengan 401, PVMBG v1 tanpa dan v2 dengan `confidence_level`, ID laporan tetap sama setelah toggle, serta respons 503 saat outage. Tes PostgreSQL membuktikan ID hazard tetap, revisi dan outbox tidak bertambah pada payload identik, dan snapshot riwayat sebelum/sesudah field baru tetap dapat dibaca. Ini merupakan bukti komponen pada commit `19beead`, bukan klaim bahwa seluruh acceptance P1 sudah diuji end-to-end.

4.5.1 Bukti Integrasi

Evidence berikut berasal dari trial dan tangkapan command yang disertakan dalam repository [10]. Trial 5 Oktober memakai commit `071b32d`; command 7 Oktober menyediakan contoh pemetaan dan audit schema tambahan.

Pada contoh BMKG 7 Oktober, magnitude 4,2 tanpa potensi tsunami menjadi severity NORMAL; region dan koordinat dipertahankan pada field kanonik. Contoh PVMBG v1 mempunyai alert Normal, menjadi VOLCANIC/NORMAL, dan ID Red Mountain dipetakan ke koordinat sintetis $(-7, 108)$. Source reference pada masing-masing hasil tetap menunjuk record sumber yang dipilih. Log validasi draft mencatat delapan record HazardEvent valid: empat dari trial 5 Oktober dan empat dari command 7 Oktober. Pernyataan ini mengenai record yang diperiksa, bukan seluruh kemungkinan input.

Pengujian schema memakai `POST /admin/schema-version` dengan versi 2, lalu membaca `history-audit -revisions 1,2` melalui binary Aggregator. Revisi pertama tidak memiliki confidence; revisi kedua memiliki nilai 0,58. Keduanya mempertahankan ID hazard dan source reference yang sama. Daftar migrasi tetap dua migrasi awal; confidence disimpan sebagai tambahan atribut JSONB.

```
$ docker compose exec -T aggregator /aggregator history-audit \
>   --source PVMBG --source-ref-id "$PVMBG_REF" --revisions 1,2 \
>   | tee docs/report/evidence/schema-command-audit.json \
>   | jq '{hazard_id, source_ref_id,
>       versions: [.versions[] | {revision, attributes: .data.attributes}],
>       migrations: [.migrations[].version]}'
>
{
  "hazard_id": "a3e0c470-1b79-4f39-aeb1-c7a063568995",
  "source_ref_id": "sim-19ac08b49fd13778-report-001",
  "versions": [
    {
      "revision": 1,
      "attributes": {
        "volcano_id": "sim-tamriel-red-mountain",
        "eruption_count_24h": 1,
        "ash_column_height_m": 125
      }
    },
    {
      "revision": 2,
      "attributes": {
        "volcano_id": "sim-tamriel-red-mountain",
        "confidence_level": 0.58,
        "eruption_count_24h": 1,
        "ash_column_height_m": 125
      }
    }
  ],
  "migrations": [
    "001_canonical_store.sql",
    "002_outbox_unpublished_index.sql"
  ]
}
```

Gambar 4.1: Audit schema dari command pada 7 Oktober: identitas hazard tetap, confidence muncul pada revisi kedua, dan daftar migrasi tidak bertambah [10].

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'sort_by(.Name) | map({name: .Name, id: .Id,
>       started_at: .State.StartedAt, restarts: .RestartCount})' >
/tmp/report-session/schema-after.json
> diff -s /tmp/report-session/schema-before.json \
>   /tmp/report-session/schema-after.json
Files /tmp/report-session/schema-before.json and /tmp/report-session/schema-
after.json are identical
```

Gambar 4.2: Perbandingan ID, waktu mulai, dan restart count container sebelum/sesudah toggle menghasilkan file identik pada sesi pengujian [10].

Hasil audit dan perbandingan container mendukung bahwa tambahan confidence tersimpan tanpa migrasi baru atau restart pada sesi tersebut. Keduanya melengkapi tes komponen, tetapi tidak menggantikan pengujian volume besar atau warning terlambat sebagai skenario tersendiri.

4.5.2 Pemetaan Bukti ke Kriteria

Kriteria	Bukti dan batas interpretasi
P1.1	Pemetaan kedua sumber: tes mapper, contoh command dan trial.
P1.2–3	Confidence v2, audit dua revisi, daftar migrasi, dan kondisi container pada Gambar 4.1–4.2.
P1.4	Field tambahan/nested dan collision: tes mapper; validasi kontrak delapan record pada log pengujian.
P1.5	Mekanisme, alternatif decoding/snapshot, dan batas perubahan aditif dijelaskan pada bab ini.

4.5.3 Temuan pada Volume Lebih Besar

Catatan review historis melaporkan toggle v2 pada stack yang telah menyimpan sekitar 1.000 record sumber. Respons mock sudah berubah, tetapi revisi baru untuk record yang diperiksa belum tersimpan; log Aggregator berulang kali menunjukkan `POLL_FAILED`. Mengembalikan schema ke v1 memulihkan polling/publication. Catatan ini bukan pengukuran load baru dan tidak dibantah oleh keberhasilan trial dengan seed awal [10].

Deadline write 2 detik diduga berpengaruh ketika satu snapshot memperbarui banyak record, tetapi penyebabnya belum dipastikan. Tidak ada perubahan kode atau timeout dalam catatan tersebut. Sebelum menyatakan dukungan schema pada volume yang terus tumbuh, perlu diagnosis serta pengukuran ulang; pagination, change feed, atau batch lebih kecil adalah opsi evaluasi, bukan fitur yang sudah diterapkan.

PROBLEM 2

CONCURRENCY, AVAILABILITY, DAN GRACEFUL DEGRADATION

Bukti runtime pada bab ini mencakup trial 5 Oktober dan tangkapan command 7 Oktober 2026, yang merupakan sesi pengujian berbeda [10].

5.1. Isolasi request dan polling

Jalur request client dipisahkan dari pengambilan data sumber. Client API membaca data yang telah tersimpan melalui Aggregator, sehingga request tidak harus menunggu mock PVMBG yang dapat terlambat 3 detik. Sementara itu, loop polling BMKG dan PVMBG berjalan sendiri-sendiri; fetch seismic dan warning BMKG juga dapat dilakukan paralel. HTTP client menggunakan kembali koneksi untuk mengurangi biaya pembukaan koneksi pada request berikutnya.

Di database, pekerjaan baca memiliki pool 10 koneksi, sedangkan penulisan BMKG, penulisan PVMBG, dan relay masing-masing mendapat 2 koneksi. Dengan pemisahan ini, pekerjaan polling tidak mengambil seluruh koneksi yang dibutuhkan jalur baca. API dan Aggregator membatasi jumlah request yang sedang diproses menjadi 128. Jika batas tersebut tercapai, service mengembalikan 429 beserta `Retry-After`, sehingga client mengetahui bahwa request perlu dicoba lagi.

Timeout membatasi lamanya setiap pekerjaan: baca database 200 ms, write 2 detik, request API ke Aggregator 1 detik, request ke BMKG 1 detik, dan request ke PVMBG 4 detik. Angka PVMBG dibuat lebih panjang agar delay 3 detik pada pengujian masih dapat ditangani.

Alternatifnya adalah mengambil data sumber untuk setiap request client. Cara itu dapat memberikan data yang lebih baru, tetapi waktu respons dan kegagalan sumber langsung memengaruhi client. Polling menghindari ketergantungan tersebut dengan konsekuensi adanya jeda pembaruan. Karena itu, API menyertakan metadata freshness agar client mengetahui status sumber dan waktu pembaruan terakhir.

5.2. Outage dan pemulihan

Ketika polling PVMBG gagal, Aggregator mempertahankan data terakhir yang berhasil disimpan. API masih dapat mengembalikannya, tetapi `meta.sources.PVMBG.status` berubah menjadi `stale`. Respons juga menyertakan `last_success_at` dan `stale_since`, sehingga client mengetahui kapan sumber terakhir berhasil dibaca dan sejak kapan data dinyatakan tertinggal. Pada saat yang sama, BMKG dapat tetap berstatus `fresh` karena polling-nya terpisah.

Jika sumber belum pernah menghasilkan data yang dapat dilayani, API menyatakan `unavailable` dengan status 503. Setelah outage dinonaktifkan, polling berikutnya memperbarui data dan mengembalikan status PVMBG menjadi fresh tanpa restart

API maupun Aggregator. Pengujian outage dilakukan melalui endpoint kontrol mock. Request berikut mengirim `enabled=true` ke `/admin/outage`; setelah respons diterima, pengujian menunggu polling mendeteksi kegagalan sumber.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"enabled":true}' \
>   http://127.0.0.1:8282/admin/outage | jq .
{
  "enabled": true
}

$ sleep 10
```

Gambar 5.1: Mengaktifkan outage pada sumber PVMBG.

Berikutnya, dua request dikirim ke Client API dengan token Tim Lapangan. Request pertama memilih PVMBG dan menampilkan jumlah record beserta metadata sumber, sedangkan request kedua membaca metadata BMKG. Pada hasil PVMBG masih ada lima record yang dikembalikan, tetapi statusnya stale dan waktu kegagalan tercatat. BMKG tetap fresh. API tetap menyediakan data terakhir beserta status dan waktu kegagalan sumber.

```
$ FIELD_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-session/field-
team.json)
> curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=PVMBG&view=full&limit=5' \
>   | jq '{records: (.data | length), source: .meta.sources.PVMBG}'
{
  "records": 5,
  "source": {
    "status": "stale",
    "last_success_at": "2026-10-07T05:15:29.209971Z",
    "stale_since": "2026-10-07T05:15:31.341025Z"
  }
}

$ curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1' \
>   | jq '.meta.sources.BMKG'
{
  "status": "fresh",
  "last_success_at": "2026-10-07T05:15:36.44186Z",
  "stale_since": null
}
```

Gambar 5.2: Respons API selama PVMBG outage: data terakhir tersedia dan kondisi setiap sumber dinyatakan secara eksplisit.

Pemulihan dilakukan dengan mengirim `enabled=false` ke endpoint yang sama. Setelah jeda polling, request API menunjukkan PVMBG kembali fresh dan `stale_since` menjadi null. Langkah ini tidak menjalankan command restart. Trial lengkap pada 5 Oktober juga membandingkan kondisi container sebelum dan sesudah outage; hasilnya tersimpan di `outage-recovery.json`.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' --data '{"enabled":false}' \
>   http://127.0.0.1:8282/admin/outage | jq .
{
  "enabled": false
}

$ sleep 10

$ curl -sS -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=PVMBG&view=full&limit=1' \
>   | jq '.meta.sources.PVMBG'
{
  "status": "fresh",
  "last_success_at": "2026-10-07T05:15:47.835933Z",
  "stale_since": null
}
```

Gambar 5.3: Mematikan outage dan membaca hasil polling yang sudah pulih.

5.3. Metode dan hasil load test

Load test dijalankan pada 5 Oktober 2026 menggunakan `scripts/load/hazards-monotonic.py`. Script ini mengirim request HTTP dengan 30 worker BMKG dan 30 worker PVMBG secara bersamaan selama 90 detik. Tidak ada think time, sehingga setiap worker langsung mengirim request berikutnya setelah menerima respons.

Setiap worker mempunyai sesi Tim Lapangan sendiri, menggunakan HTTP keep-alive, dan membaca `view=full` dengan limit 50. Access token tetap menggunakan TTL 60 detik; script melakukan refresh sebelum expiry agar masa berlaku token tidak menjadi penyebab putusnya beban. Selama pengujian, delay sumber PVMBG ditetapkan 3000 ms.

Latency dihitung sejak request dimulai hingga seluruh body diterima, termasuk pembukaan koneksi jika diperlukan. Pengukuran memakai `time.perf_counter_ns`, yaitu timer monotonic yang tidak mengikuti perubahan wall clock host, sedangkan percentile dihitung dengan metode nearest rank.

Untuk memeriksa syarat koneksi paralel, collector menggunakan Linux `ss` dan mengambil sampel koneksi TCP established ke port API setiap sekitar 1 detik. Pengukuran ini diperlukan karena jumlah worker saja tidak membuktikan jumlah koneksi yang benar-benar terbuka.

Sebelum beban diberikan, delay mock diatur dengan body `min_ms` dan `max_ms` yang sama-sama bernilai 3000. Request langsung ke sumber kemudian mengukur waktu respons melalui `curl`. Flag `-o /dev/null` membuang body, sedangkan `-w` menampilkan status HTTP dan durasi request. Ini membedakan delay sumber dari latency Client API yang membaca data tersimpan.

```
$ curl -sS --fail-with-body \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   -H 'Content-Type: application/json' \
>   --data '{"min_ms":3000,"max_ms":3000}' \
>   http://127.0.0.1:8282/admin/delay | jq .
> curl -sS -o /dev/null -w 'HTTP %{http_code}; durasi %{time_total} s\n' \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports
{
  "min_ms": 3000,
  "max_ms": 3000
}
HTTP 200; durasi 3.018729 s
```

Gambar 5.4: Mengatur delay 3 detik dan mengukur request langsung ke sumber PVMBG.

Collector TCP dijalankan bersamaan dengan load test untuk mengukur jumlah koneksi established selama beban berlangsung. Kedua proses dimulai dari root repo dengan command berikut:

```
export API_URL=http://127.0.0.1:8280
export AUTH_URL=http://127.0.0.1:8284
python3 scripts/load/capture-tcp-connections.py \
  --api-url "$API_URL" --seconds 110 --interval 1 \
  --output /tmp/report-tcp.json &
tcp_pid=$!
python3 scripts/load/hazards-monotonic.py \
  --seconds 90 --vus-per-source 30 --output /tmp/report-load.json
wait "$tcp_pid"
```

Hasil load test tersimpan di `docs/report/evidence/load.json`. Field yang diperiksa meliputi jumlah worker, durasi pengujian, p95 BMKG, throughput, dan persentase error.

```
$ jq '{started_at, profile, elapsed_seconds,
>     bmkg_p95_ms: .sources.BMKG.success_duration_ms.p95,
>     throughput: .hazard_successes_per_second,
>     error_percent: .hazard_unexpected_error_percent}' \
>   docs/report/evidence/load.json
{
  "started_at": "2026-10-05T13:11:41.596066+00:00",
  "profile": {
    "vus_per_source": 30,
    "duration_seconds": 90.0,
    "view": "full",
    "limit": 50,
    "identity": "field-team",
    "access_ttl_seconds": 60
  },
  "elapsed_seconds": 90.08667208399856,
  "bmkg_p95_ms": 124.521589,
  "throughput": 980.25599078252,
  "error_percent": 0.03396047001290498
}
```

Gambar 5.5: Membaca hasil load test 5 Oktober dari berkas pengukuran yang disimpan.

Metrik	BMKG	PVMBG
Request terkirim	44.510	43.828
Respons sukses	44.494	43.814
Error selain 429	16	14
HTTP 429 pada API	0	0
p50 sukses (ms)	52,07	52,64
p95 sukses (ms)	124,52	125,24
p99 sukses (ms)	190,71	194,30
p95 seluruh respons (ms)	124,76	125,44

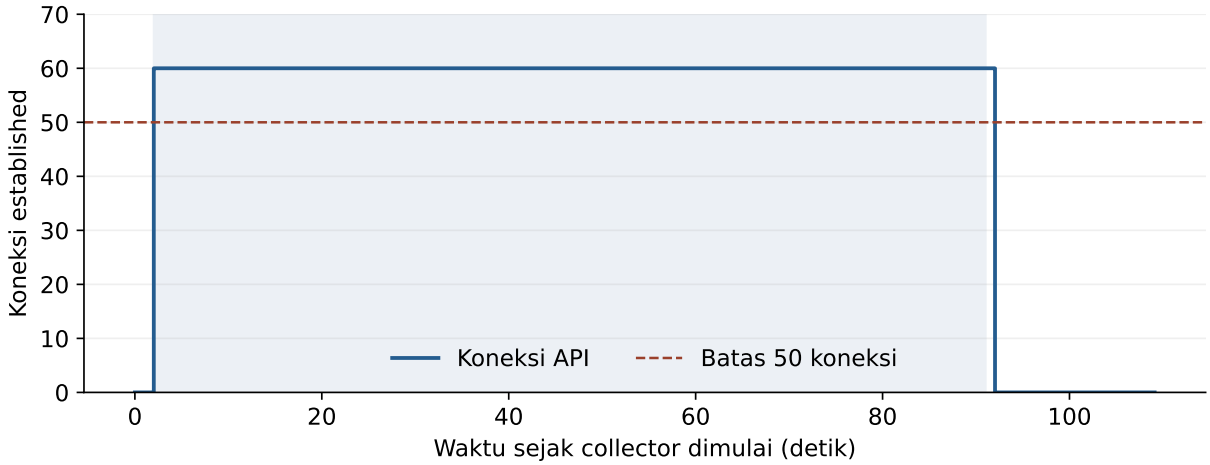
Tabel 5.1: Hasil beban simultan selama 90,09 detik. p50/p95/p99 sukses dihitung dari 2xx; p95 seluruh respons juga ditampilkan.

Dari 88.338 request yang dikirim, 88.308 mendapat respons sukses. Throughput-nya mencapai 980,26 respons sukses per detik, sedangkan 30 request lainnya menghasilkan HTTP 503, sebagaimana dicatat dalam `docs/report/evidence/load-statuses.json`. Dengan demikian, error selain 429 sebesar 0,034%, masih di bawah batas 1%.

API tidak menghasilkan 429 pada pengujian ini. Auth menghasilkan 18 respons 429 selama login dan refresh, tetapi retry berhasil sehingga tidak ada sesi worker yang

gagal. Angka Auth tersebut dilaporkan terpisah agar pembatasan request terkendali tidak tercampur dengan error pada pembacaan hazard.

Delay langsung dari sumber PVMBG terukur 3.004,46 ms, sementara p95 respons sukses BMKG sebesar 124,52 ms. Hasil ini memenuhi batas 300 ms ketika kedua sumber dibaca bersamaan. Sampel TCP juga menunjukkan sedikitnya 50 koneksi pada interval 89,00 detik, melebihi syarat 60 detik. Karena collector mengambil sampel sekitar sekali per detik, kesimpulan mengenai koneksi mengikuti resolusi pengukuran tersebut.



Gambar 5.6: Sampel koneksi established selama pengujian. Garis putus-putus menunjukkan batas 50 koneksi. Interval di atas batas dihitung dari rangkaian sampel koneksi.

Kriteria	Bukti
P2.1	Delay sumber terukur dan BMKG p95 pada Tabel 5.1; kedua sumber menerima request bersamaan.
P2.2	Throughput, percentile, error, 429, serta sampel TCP; Tabel 5.1 dan Gambar 5.6.
P2.3	Kontrol outage, data stale, dan pemulihan pada Gambar 5.1–5.3; trial lengkap di `outage-recovery.json`.
P2.4	Metode, konfigurasi, serta mekanisme isolasi di bab ini; script dan output lengkap disertakan.

5.4. Batasan

Hasil load test menggambarkan beban yang dicoba pada satu host dengan pembacaan maksimal 50 record per request. Pengujian tersebut belum mencari kapasitas maksimum sistem. Selain itu, jalur baca tetap membutuhkan Aggregator dan PostgreSQL, sehingga isolasi dari gangguan sumber tidak menjamin API tersedia jika salah satu komponen ini mati.

Penilaian freshness masih menggunakan wall clock. Jika waktu host bergeser, umur data yang ditampilkan dapat ikut berubah meskipun pengukuran latency tetap benar. Perluasan ke beberapa node karena itu memerlukan sinkronisasi waktu serta pengujian ulang terhadap kapasitas dan pola kegagalan yang baru.

PROBLEM 3

SERVICE-TO-SERVICE AUTHENTICATION DAN TRUST

Bukti runtime pada bab ini mencakup trial 5 Oktober dan tangkapan command 7 Oktober 2026, yang merupakan sesi pengujian berbeda [10].

6.1. Credential dan hak akses

Setiap sumber memiliki credential sendiri: BMKG menerima API key melalui header `X-BMKG-Key`, sedangkan PVMBG menerima Bearer token sumber. API internal Aggregator memakai key lain. Pemisahan ini membuat kebocoran satu credential tidak otomatis memberikan akses ke seluruh antarmuka, dan credential sumber tidak dapat dipakai sebagai identitas client.

Untuk akses client, Auth membedakan identitas `media`, `field-team`, dan `bnpb-internal`. Media hanya memiliki scope ringkasan, yang berisi tepat tujuh field: `hazard_id`, `source`, `hazard_type`, `severity`, `area_name`, `occurred_at`, dan `ingested_at`. Scope lengkap menambahkan `source_ref_id`, koordinat, dan `attributes`.

Pembatasan dilakukan oleh Client API sebelum data diminta dari Aggregator. Jadi, Media tidak dapat memperoleh data raw hanya dengan mengubah query menjadi `view=full` atau meminta field tambahan secara eksplisit; kedua request tersebut ditolak dengan 403. Aturan akses berada di server, sehingga tidak bergantung pada tampilan yang dipakai client.

JWT ditandatangani Auth menggunakan RS256 dan private key. API hanya memerlukan public key untuk memeriksa signature, issuer, audience, expiry, serta scope. Verifikasi dilakukan secara lokal, sehingga token yang masih valid tetap dapat digunakan saat Auth berhenti sementara [11]. Jika API memakai shared signing secret, material untuk memverifikasi juga dapat digunakan untuk membuat token. Sementara itu, introspection ke Auth pada setiap request memudahkan pemeriksaan status token secara terpusat, tetapi menambahkan dependency pada jalur baca. Untuk milestone 1, verifikasi lokal dipilih bersama TTL yang singkat.

6.2. Expiry dan refresh

Access token berlaku 60 detik. Agar client dapat melanjutkan sesi tanpa login ulang, Auth juga memberikan refresh token berupa nilai acak. Hanya hash token tersebut yang disimpan di SQLite. Setiap refresh menghasilkan token baru dan menandai token refresh sebelumnya sudah digunakan. Jika token lama dipakai kembali, keluarga sesi itu dibatalkan untuk mencegah kelanjutan refresh dari token yang digunakan ulang.

Batas umur keluarga sesi adalah 24 jam dan tetap dihitung dari pembentukan sesi, sehingga rotasi tidak memperpanjang sesi tanpa batas. Database Auth berada pada volume privat agar sesi refresh tidak hilang ketika container dijalankan ulang.

Rotasi refresh token tidak langsung mencabut access JWT yang belum expired. API tidak memiliki daftar revocation untuk access token; token tersebut masih dapat digunakan sampai `exp`. Pengujian menunggu expiry alami sebelum memeriksa penolakan token lama dengan status 401.

6.3. Pengujian credential dan hak akses

6.3.1 Credential hanya berlaku pada sumbernya

Pengujian credential dilakukan sebagai empat request terpisah. Untuk BMKG, header tetap bernama `X-BMKG-Key`: request pertama mengisinya dengan key BMKG, sedangkan request kedua memakai token PVMBG sebagai nilai yang salah. Status HTTP setiap request dibaca melalui opsi `-w`.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "X-BMKG-Key: $BMKG_API_KEY" \
>   http://127.0.0.1:8281/seismic-events
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "X-BMKG-Key: $PVMBG_TOKEN" \
>   http://127.0.0.1:8281/seismic-events
HTTP 401
```

Gambar 6.1: BMKG menerima key miliknya dengan 200 dan menolak credential PVMBG dengan 401.

Arah sebaliknya diuji pada PVMBG dengan skema Bearer yang sama untuk kedua request. Token PVMBG menghasilkan 200, sedangkan key BMKG menghasilkan 401. Karena format header tetap sesuai dengan antarmuka sumber, status 401 menunjukkan bahwa sumber menolak nilai credential yang salah.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $PVMBG_TOKEN" \
>   http://127.0.0.1:8282/volcanic-reports
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $BMKG_API_KEY" \
>   http://127.0.0.1:8282/volcanic-reports
HTTP 401
```

Gambar 6.2: PVMBG menerima token miliknya dan menolak key BMKG.

6.3.2 Media menerima ringkasan dan tidak dapat meminta data raw

Media terlebih dahulu login ke Auth dengan identitas dan secret client-nya sendiri. Respons login disimpan pada file privat untuk digunakan pada request berikutnya. Scope `hazards:summary` menjadi dasar pembatasan di API.

```
$ umask 077
> curl -sS --fail-with-body \
>   -H 'Content-Type: application/json' \
>   --data
"{\"client_id\":\"media\",\"client_secret\":\"$MEDIA_CLIENT_SECRET\"}" \
>   http://127.0.0.1:8284/v1/auth/login -o .secrets/report-
session/media.json
> jq '{token_type, expires_in, scope}' .secrets/report-session/media.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary"
}
```

Gambar 6.3: Auth memberikan scope ringkasan dan TTL 60 detik untuk Media.

Access token dari respons login dipasang pada header Authorization ketika Media membaca satu record BMKG. Filter `keys` menampilkan daftar field yang diterima: jumlahnya tepat tujuh dan tidak mencakup koordinat, `source_ref_id`, maupun `attributes`.

```
$ MEDIA_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-
session/media.json)
> curl -sS -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&limit=1' \
>   | jq '.data[0] | keys'
[
  "area_name",
  "hazard_id",
  "hazard_type",
  "ingested_at",
  "occurred_at",
  "severity",
  "source"
]
```

Gambar 6.4: Field respons Media sesuai dengan kontrak ringkasan.

Media kemudian mencoba `view=full` dan `fields=attributes` melalui dua request tersendiri. Keduanya ditolak dengan 403. Hasil ini menunjukkan bahwa query yang diubah client tidak dapat melewati pembatasan server.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?view=full'
HTTP 403

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $MEDIA_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?fields=attributes'
HTTP 403
```

Gambar 6.5: Dua cara meminta data raw sama-sama ditolak untuk token Media.

6.4. Pengujian expiry dan refresh

Login Tim Lapangan menghasilkan scope `hazards:summary hazards:raw` dan access token berumur 60 detik. Access token dari respons login digunakan pada request API berikutnya.

```
$ umask 077
> curl -sS --fail-with-body \
>   -H 'Content-Type: application/json' \
>   --data "{\"client_id\":\"field-
team\",\"client_secret\":\"$FIELD_TEAM_CLIENT_SECRET\"}" \
>   http://127.0.0.1:8284/v1/auth/login -o .secrets/report-session/field-
team.json
> jq '{token_type, expires_in, scope}' .secrets/report-session/field-
team.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary hazards:raw"
}
```

Gambar 6.6: Login Tim Lapangan memberikan izin ringkasan dan data raw.

Command berikut membaca payload JWT untuk menampilkan `iat` dan `exp`, kemudian menghitung selisihnya. Decoding pada terminal hanya digunakan untuk inspeksi claim; verifikasi token tetap dilakukan API ketika request dikirim. Selisihnya 60 detik dan token yang belum expired memperoleh 200.

```
$ FIELD_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-session/field-
team.json)
> jq -r .access_token .secrets/report-session/field-team.json \
>   | jq -R 'split(".")[1] | @base64d | fromjson | {iat, exp, ttl: (.exp-
.iat)}'
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
{
  "iat": 1791349941,
  "exp": 1791350001,
  "ttl": 60
}
HTTP 200
```

Gambar 6.7: Memeriksa TTL dari claim dan menggunakan access token sebelum expiry.

Selanjutnya, loop menunggu wall clock melewati `exp` dari token yang sama. Request API setelah batas tersebut menghasilkan 401.

```
$ ACCESS_EXP=$(jq -r .access_token .secrets/report-session/field-team.json \
>   | jq -R 'split(".")[1] | @base64d | fromjson | .exp')
> while [ "$(date +%s)" -le "$ACCESS_EXP" ]; do sleep 1; done
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 401
```

Gambar 6.8: Token lama ditolak setelah waktu expiry terlewati.

Refresh dilakukan dengan mengambil `refresh_token` dari file login dan mengirimkannya sebagai body JSON ke Auth. Opsi `-data-binary @-` mengirim body JSON dari pipeline, yang berisi refresh token dari respons login. Respons baru disimpan pada file lain dan menunjukkan TTL 60 detik serta scope yang sama. Tidak ada request login pada langkah ini.

```
$ jq '{refresh_token}' .secrets/report-session/field-team.json \
>   | curl -sS --fail-with-body -H 'Content-Type: application/json' \
>       --data-binary @- http://127.0.0.1:8284/v1/auth/refresh \
>       -o .secrets/report-session/refreshed.json
> jq '{token_type, expires_in, scope}' .secrets/report-
session/refreshed.json
{
  "token_type": "Bearer",
  "expires_in": 60,
  "scope": "hazards:summary hazards:raw"
}
```

Gambar 6.9: Mendapatkan token baru melalui refresh tanpa login ulang.

Token baru kemudian mendapat 200 ketika digunakan ke API, sementara token lama tetap mendapat 401. Trial integrasi 5 Oktober juga memeriksa sesi refresh setelah restart Auth, replay refresh token, dan penggunaan JWT yang masih valid ketika Auth berhenti; bukti tambahannya ada di `integration.json`.

```
$ NEW_ACCESS_TOKEN=$(jq -r .access_token .secrets/report-
session/refreshed.json)
> curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $NEW_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 200

$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 401
```

Gambar 6.10: Token baru dapat digunakan, sedangkan access token lama tetap expired.

Kriteria	Bukti
P3.1	Empat request pada Gambar 6.1 dan 6.2.
P3.2	Tujuh field Media dan penolakan 403 pada Gambar 6.4 serta 6.5.
P3.3	TTL, expiry alami, refresh, serta pemeriksaan token lama dan baru pada Gambar 6.7–6.10.
P3.4	Secret pada `.env.example` kosong; `.env` dan `.secrets/` diabaikan Git serta dikecualikan dari build context. Key dan sesi tersimpan pada volume privat.
P3.5	Pilihan autentikasi, trade-off, dan batas keamanan dijelaskan pada bab ini.

6.5. Menelusuri request dengan correlation ID

Request dapat diberi header `X-Correlation-ID` agar jejaknya dapat dicari di log API dan Aggregator. Pada contoh berikut, ID ditentukan oleh client dan diteruskan ke Aggregator. ID ini menghubungkan catatan dari request yang sama pada kedua service.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $NEW_ACCESS_TOKEN" \
>   -H 'X-Correlation-ID: report-command-correlation' \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
>
HTTP 200
```

Gambar 6.11: Request client dengan correlation ID yang ditentukan secara eksplisit.

Command log kemudian memilih catatan dengan ID tersebut. Keluaran menunjukkan operasi API, pembacaan upstream, dan operasi Aggregator yang semuanya memiliki correlation ID sama.

```
$ docker compose logs --no-color --no-log-prefix --since 2m \
>   client-api aggregator \
>   | jq -R 'fromjson? | select(.correlation_id == "report-command-
correlation")
>       | {service, correlation_id, operation, status, duration_ms}'
>
{
  "service": "aggregator",
  "correlation_id": "report-command-correlation",
  "operation": "GET /internal/v1/hazards",
  "status": 200,
  "duration_ms": 2
}
{
  "service": "client-api",
  "correlation_id": "report-command-correlation",
  "operation": "aggregator_read",
  "status": 200,
  "duration_ms": 3
}
{
  "service": "client-api",
  "correlation_id": "report-command-correlation",
  "operation": "GET /v1/hazards",
  "status": 200,
  "duration_ms": 4
}
```

Gambar 6.12: Log kedua service dapat ditelusuri melalui correlation ID yang sama.

6.6. Batasan keamanan dan waktu

Expiry JWT mengikuti wall clock. Pada implementasi ini, `iat` yang tampak berada di masa depan ketika clock mundur tidak dijadikan alasan tunggal untuk menolak token. API tetap memeriksa signature, issuer, audience, scope, expiry, serta `nbf` jika claim tersebut ada. Selisih `exp-iat` tetap 60 detik, tetapi pergeseran waktu host dapat mengubah kapan batas itu tercapai: clock mundur dapat memperpanjang masa berlaku yang dirasakan, sedangkan clock maju dapat membuat token lebih cepat expired. Sinkronisasi waktu diperlukan agar TTL sesuai dengan waktu nyata.

Di luar masalah waktu, komunikasi HTTP internal belum memakai TLS dan domain event masih menggunakan token NATS bersama. Sistem juga belum menyediakan rotasi key RSA maupun pencabutan access JWT secara langsung. Sebelum digunakan pada jaringan yang tidak dipercaya, perlindungan koneksi dan pemisahan credential broker perlu ditambahkan.

PROBLEM 4

MICROSERVICE DAN FLEXIBLE STORAGE

Bukti runtime pada bab ini mencakup trial 5 Oktober dan tangkapan command 7 Oktober 2026, yang merupakan sesi pengujian berbeda [10].

7.1. Container dan ownership

Stack utama terdiri dari tujuh container aplikasi serta dua container infrastruktur, yaitu PostgreSQL dan NATS. Setiap aplikasi memiliki Dockerfile sendiri, sementara kedua komponen infrastruktur memakai image resmi. Third Consumer hanya dijalankan melalui profile demo. Compose membantu mengatur jaringan dan urutan startup, tetapi satu aplikasi tetap dapat dibangun dan diganti dengan `-no-deps` tanpa menjalankan ulang seluruh stack.

Canonical Store menjadi tanggung jawab Aggregator. Hanya Aggregator dan PostgreSQL yang terhubung ke `store-net`, dan PostgreSQL tidak membuka host port. Karena itu, Client API membaca data melalui HTTP Aggregator. Consumer memperoleh data melalui NATS, lalu menyimpan hasil olahan dalam database lokalnya sendiri. Perintah audit juga dijalankan melalui binary Aggregator agar inspeksi data tetap dilakukan oleh service pemiliknya.

Pembagian tersebut menghindari ketergantungan consumer pada schema tabel internal Aggregator. Sebagai konsekuensinya, tampilan consumer diperbarui secara eventual: perubahan canonical harus melewati outbox dan broker sebelum diterapkan pada database consumer. Shared database antarservice dapat mengurangi jumlah antarmuka HTTP, tetapi perubahan tabel akan mengikat beberapa aplikasi sekaligus dan tidak memenuhi ownership storage yang diminta spesifikasi.

7.2. Detail Implementasi Canonical Store

Aggregator menggunakan PostgreSQL dengan empat kelompok tabel. `hazard_events` menyimpan state terbaru dan unique key sumber/reference; `hazard_versions` menyimpan snapshot append-only per revisi; `source_state` menyimpan percobaan, sukses, kegagalan, dan freshness; `outbox` menyimpan envelope immutable yang belum atau sudah dipublikasikan. Field kanonik memiliki tipe dan constraint, sedangkan atribut sumber memakai JSONB [5].

Perubahan latest hazard, penambahan versi, dan outbox ditulis dalam satu transaksi. Jika isi identik secara semantik, termasuk angka JSON yang ekuivalen, revisi tidak ditambah. Kapasitas pending outbox diperiksa secara atomik; snapshot yang melampaui batas tidak di-commit. Pool baca dipisahkan dari writer BMKG, writer PVMBG, dan relay sehingga request baca tidak menunggu delay sumber.

Migrasi versioned dijalankan Aggregator saat startup dengan advisory lock dan checksum. Schema v2 PVMBG hanya menambah key JSONB, bukan migrasi baru. CLI

audit pada binary Aggregator membaca versi terpilih melalui adapter Store lokal; tidak tersedia endpoint history publik atau akses database untuk consumer.

7.3. Struktur Storage dan Alternatif

Field canonical yang stabil disimpan dalam kolom bertipe dengan constraint, sedangkan `attributes` memakai JSONB untuk menampung atribut yang dapat bertambah. Data terbaru berada di `hazard_events` dan setiap revisinya dicatat dalam `hazard_versions`. Dua tabel lain melengkapi alur ingest: `source_state` menyimpan kondisi polling, sementara `outbox` mencatat pesan yang perlu dipublikasikan.

Dengan struktur ini, tambahan `confidence_level` masuk ke dokumen atribut tanpa `ALTER TABLE`. Pilihan tersebut dibandingkan dengan dua alternatif pada tabel berikut.

Pilihan	Kelebihan	Konsekuensi
PostgreSQL + JSONB (dipilih)	Constraint identitas, query canonical, dan transaksi outbox dalam satu store	Atribut dinamis tetap perlu validasi; query JSONB yang sering dipakai dapat membutuhkan index tambahan.
Relasional dengan kolom tetap	Tipe setiap atribut jelas dan mudah di-query	Field baru membutuhkan migrasi; kurang sesuai perubahan schema langsung pada P4.
Document store, misalnya MongoDB	Bentuk dokumen sumber mudah disimpan dan berubah	Aturan canonical, index, dan transaksi history/outbox tetap perlu dirancang; fleksibilitas dokumen saja tidak menjamin konsistensi.

Tabel 7.1: Perbandingan pilihan penyimpanan. Fleksibilitas dokumen tidak menggantikan validasi kontrak.

7.4. Pengujian penggantian satu container

Pengujian menyimpan ID, waktu mulai, dan restart count container selain Dashboard sebelum melakukan rebuild. Dashboard dikecualikan dari daftar ini karena container itulah yang akan diganti.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'map(select(.Config.Labels["com.docker.compose.service"]
>       != "dashboard-consumer")) | sort_by(.Name)
>       | map({name: .Name, id: .Id, started_at: .State.StartedAt,
>           restarts: .RestartCount})' > .secrets/report-session/rebuild-
before.json
```

Gambar 7.1: Mencatat kondisi container lain sebelum Dashboard dibangun ulang.

Dashboard kemudian dihentikan dan image-nya dibangun kembali. Log build disimpan ke file, lalu `tail -n 6` membaca bagian akhir yang menunjukkan image berhasil dibangun. Log lengkap tersedia pada `evidence/dashboard-rebuild.log`.

```
$ docker compose stop dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Stopping
 Container aat-report-revision-dashboard-consumer-1  Stopped

$ docker compose build dashboard-consumer \
>   > .secrets/report-session/rebuild.log 2>&1
> tail -n 6 .secrets/report-session/rebuild.log

#15 [dashboard-consumer] exporting to image
#15 exporting layers done
#15 writing image
sha256:c54f9851d0aa3b832644178aac40303d6fd0d5e90ffaaecaa5989e5071680245 done
#15 naming to docker.io/library/aat-report-revision-dashboard-consumer done
#15 DONE 0.0s
```

Gambar 7.2: Menghentikan Dashboard dan membangun image service tersebut.

Saat Dashboard masih berhenti, request BMKG ke API menghasilkan 200. Dashboard kemudian dijalankan kembali dengan `-no-deps -force-recreate`; opsi pertama menjaga dependency tidak ikut dijalankan ulang, sedangkan opsi kedua memastikan container Dashboard diganti. `-wait` menunggu hasil healthcheck-nya siap.

```
$ curl -sS -o /dev/null -w 'HTTP %{http_code}\n' \
>   -H "Authorization: Bearer $FIELD_ACCESS_TOKEN" \
>   'http://127.0.0.1:8280/v1/hazards?source=BMKG&view=full&limit=1'
HTTP 200

$ docker compose up -d --no-deps --force-recreate --wait dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Recreate
 Container aat-report-revision-dashboard-consumer-1  Recreated
 Container aat-report-revision-dashboard-consumer-1  Starting
 Container aat-report-revision-dashboard-consumer-1  Started
 Container aat-report-revision-dashboard-consumer-1  Waiting
 Container aat-report-revision-dashboard-consumer-1  Healthy
```

Gambar 7.3: API tetap merespons saat Dashboard mati, lalu Dashboard diganti secara terpisah.

Inspeksi diulang dengan proyeksi yang sama, kemudian `diff -s` membandingkan file sebelum dan sesudah. Keduanya identik dan berisi delapan container lain. Jadi, penggantian Dashboard tidak mengubah ID, waktu mulai, maupun restart count service lainnya. Trial integrasi 5 Oktober juga mencatat 10 request API sukses selama rebuild; rincian pengujian tersebut tetap tersedia di `integration.json`.

```
$ docker inspect $(docker compose ps -q) \
>   | jq -S 'map(select(.Config.Labels["com.docker.compose.service"]
>       != "dashboard-consumer")) | sort_by(.Name)
>       | map({name: .Name, id: .Id, started_at: .State.StartedAt,
>           restarts: .RestartCount})' > .secrets/report-session/rebuild-
after.json
> diff -s .secrets/report-session/rebuild-before.json \
>   .secrets/report-session/rebuild-after.json
> jq length .secrets/report-session/rebuild-after.json
Files .secrets/report-session/rebuild-before.json and .secrets/report-
session/rebuild-after.json are identical
8
```

Gambar 7.4: Delapan container lain tidak berubah setelah Dashboard diganti.

7.5. Pengujian ownership dan flexible storage

Inspeksi jaringan menunjukkan bahwa `store-net` hanya memuat Aggregator dan PostgreSQL. Inspeksi port PostgreSQL juga menunjukkan bahwa port database tidak dipublikasikan ke host. Hasil ini mendukung batas akses pada konfigurasi; kode Client API dan consumer tetap diperiksa untuk memastikan keduanya tidak membuka koneksi SQL ke Canonical Store.

```
$ docker network inspect "${COMPOSE_PROJECT_NAME}_store-net" \
>   | jq '.[0].Containers | [.[] | .Name] | sort'
> docker inspect $(docker compose ps -q postgres) \
>   --format '{{json .NetworkSettings.Ports}}'
>
[
  "aat-report-revision-aggregator-1",
  "aat-report-revision-postgres-1"
]
{}
```

Gambar 7.5: Anggota jaringan storage dan port PostgreSQL yang tidak dipublikasikan.

Fleksibilitas storage dibuktikan bersama pengujian P1 pada Gambar 4.1. Snapshot revisi 1 dan 2 tetap terbaca dengan daftar migrasi yang sama. Pembacaan tersebut dilakukan melalui `/aggregator history-audit`, sehingga perubahan dokumen diperiksa oleh service pemilik storage.

Kriteria	Bukti
P4.1	Rebuild, request API, dan perbandingan delapan container pada Gambar 7.1–7.4.
P4.2	Revisi v1/v2 dan daftar migrasi pada Gambar 4.1.
P4.3	Anggota `store-net`, port privat, serta jalur baca pada kode; Gambar 7.5.
P4.4	Batas service pada Bab 2 dan perbandingan tiga pilihan storage pada bab ini.

7.6. Verifikasi Komponen Storage

Verifikasi native pada commit `19beead`, dengan PostgreSQL 18.6 dan NATS 2.10.24, lulus untuk UUID stabil, revisi/history, idempotensi, kapasitas outbox, presisi angka, dan round-trip JSONB ke JetStream. Tes normal dan race, build, vet, serta module verify lulus. Database disposable berakhiran `_test` memakai reset opt-in, tanpa mengakses database aplikasi. Bukti ini melengkapi sesi Compose, tetapi bukan pengulangan deployment PostgreSQL 16 atau restart seluruh aplikasi.

7.7. Batasan

Pemisahan aplikasi/jaringan tidak membatasi administrator Docker. Rebuild satu consumer tidak membuktikan ketersediaan API ketika Aggregator atau PostgreSQL mati. History, inbox, dan outbox terpublikasi belum memiliki pembersihan jangka panjang; retention lokal dan batch snapshot besar pada P1 masih perlu ditangani sebelum peningkatan volume.

PROBLEM 5

FAN-OUT EVENT DISTRIBUTION / PUB-SUB

Bukti runtime pada bab ini mencakup trial 5 Oktober dan tangkapan command 7 Oktober 2026, yang merupakan sesi pengujian berbeda [10].

8.1. Producer, stream, dan consumer

Distribusi event dimulai dari transaksi penyimpanan Aggregator. Event canonical, revisinya, dan pesan outbox ditulis bersamaan, sehingga event yang berhasil disimpan mempunyai catatan untuk dikirim ke broker. Relay kemudian mempublikasikan envelope ke subject `hazard.events.v1.BMKG` atau `hazard.events.v1.PVMBG`. Kedua subject ditangkap oleh stream `HAZARD_EVENTS` melalui filter `hazard.events.v1.*`.

Outbox dipilih untuk menangani celah antara penyimpanan data dan publish. Jika publish langsung dilakukan setelah commit, proses dapat mati ketika data sudah tersimpan tetapi pesan belum terkirim. Sebaliknya, publish sebelum commit dapat mengirim event dari transaksi yang kemudian gagal. Outbox mencatat kebutuhan publish dalam transaksi data yang sama, lalu relay mencobanya kembali. Konsekuensinya, ada jeda sebelum event sampai ke consumer dan kemungkinan publish berulang.

Producer mempublikasikan pesan berdasarkan subject dan kontrak event, sehingga penambahan pembaca tidak memerlukan perubahan pada producer. Daftar consumer serta posisi baca masing-masing dikelola oleh broker.

Dashboard dan Notifier memakai durable yang berbeda, yaitu `dashboard-v1` dan `notifier-v1`. Broker menyimpan posisi baca setiap durable meskipun aplikasinya berhenti. Dengan posisi yang terpisah, masing-masing aplikasi dapat menerima seluruh event dan memberikan ACK sendiri.

Jika kedua aplikasi berbagi satu durable, pesan akan dibagi sebagai pekerjaan untuk pembaca durable tersebut. Pola itu tidak memenuhi kebutuhan fan-out, yaitu event yang sama dapat diterima oleh setiap aplikasi. Karena itu, kedua consumer memakai pull consumer yang independen meskipun stream-nya sama.

Pull dipilih agar consumer meminta batch berikutnya setelah pengolahan batch sebelumnya selesai. Dibanding push, laju pengambilan lebih langsung dikendalikan aplikasi; ukuran batch dan batas ACK membatasi pesan yang sedang ditangani. Saat stream kosong, consumer tetap melakukan fetch dengan timeout.

Konfigurasi	Nilai dan fungsi
Storage / replica / retention	File / 1 / LimitsPolicy; ACK consumer tidak menghapus event dari stream.
Batas retention	24 jam, 256 MiB, 100.000 pesan; batas yang tercapai lebih dahulu menentukan kapasitas.
Batas satu pesan	64 KiB, termasuk ukuran yang diperhitungkan publisher.
Saat kapasitas penuh	DiscardNew; publish baru ditolak, bukan menghapus pesan lama hanya untuk menerima pesan baru.
Dedup broker	Message ID `hazard_id:revision`; window 2 menit.
ACK / redelivery	AckExplicit, AckWait 30 detik, MaxDeliver -1; retry tidak dibatasi jumlahnya.
Pull / inflight	Batch 10, expiry fetch 2 detik, MaxAckPending 64.

Tabel 8.1: Konfigurasi JetStream dan consumer yang digunakan pada milestone 1.

LimitsPolicy mempertahankan event sampai batas retention, walaupun consumer telah memberikan ACK. Dengan DeliverAll, consumer baru dapat membaca event lama yang masih tersimpan. InterestPolicy juga mendukung fan-out, tetapi menghapus pesan setelah seluruh consumer yang berkepentingan memberikan ACK. Sementara itu, WorkQueuePolicy ditujukan untuk pembagian pekerjaan dengan satu pemroses per pesan [8]. LimitsPolicy dipilih agar penambahan consumer pada P5 dapat disertai replay, dengan biaya penyimpanan event selama masa retention.

Alternatif producer memanggil setiap consumer melalui HTTP membutuhkan daftar tujuan dan retry per tujuan di producer. NATS Core memberi fan-out sederhana, tetapi tidak menyimpan backlog durable untuk consumer yang mati. JetStream dipilih agar replay dan posisi baca dikelola broker. Kafka atau RabbitMQ juga memungkinkan pola sejenis, tetapi kebutuhan milestone 1 dapat dipenuhi dengan JetStream dan konfigurasi lokal yang lebih kecil.

8.2. Semantik pengiriman dan idempotensi

Semantik pengiriman sistem adalah **at-least-once**, sehingga sebuah pesan dapat diterima lebih dari sekali. Misalnya, broker sudah menerima publish, tetapi relay gagal mencatat status published di database. Relay akan mencoba kembali. Hal serupa dapat terjadi ketika consumer sudah melakukan commit lokal, tetapi proses mati sebelum ACK diterima broker. Message ID `hazard_id:revision` membantu broker mengenali publish ulang selama dedup window 2 menit, tetapi perlindungan tersebut memiliki batas waktu.

Untuk menangani pengiriman ulang, setiap consumer menyimpan `message_id` pada inbox dengan constraint unik. Pencatatan inbox dan hasil olahan dilakukan dalam satu transaksi SQLite, kemudian consumer mengirim ACK. Jika pesan yang sama datang kembali, inbox menunjukkan bahwa hasilnya sudah diterapkan sehingga side effect lokal tidak diulang. Dashboard hanya menerima revisi yang lebih baru, sedangkan Notifier menggunakan identitas pesan yang unik untuk alert-nya.

Pesan dengan envelope tidak valid disimpan ke quarantine sebelum ACK, agar pesan rusak tidak terus diulang dan payload tetap tersedia untuk diperiksa. Jika penyimpanan gagal, consumer meminta pengiriman ulang sehingga pesan belum dianggap

selesai.

Dengan demikian, hasil lokal dibuat idempoten meskipun delivery dapat berulang. Mekanisme ini tidak menyatakan bahwa seluruh alur bersifat exactly-once, terutama jika nantinya Notifier melakukan panggilan ke layanan SMS eksternal yang berada di luar transaksi SQLite.

Saat publish gagal, relay mencoba kembali dengan backoff 1 sampai 30 detik. Pending outbox dibatasi 10.000 pesan; jika batas ini tercapai, ingest tidak terus menambah antrean. Penghentian satu consumer karena itu tidak langsung menghalangi producer, selama broker masih dapat menyimpan event. Namun, kapasitas broker dan disk tetap menjadi batas: dengan DiscardNew, publish baru ditolak ketika stream penuh dan pesan tetap menunggu di outbox.

8.3. Pengujian fan-out dan recovery

8.3.1 Mencatat posisi dua durable consumer

Pengujian membaca endpoint monitoring JetStream melalui `wget` di dalam container NATS. Port monitoring tidak dibuka ke host. Query `streams=true&consumers=true` meminta informasi stream beserta consumer-nya, kemudian `jq` memilih dua durable utama.

Pada output, `last_seq` adalah sequence terakhir stream, sedangkan `ack` menampilkan ACK floor consumer dalam sequence stream. ACK floor menunjukkan posisi yang sudah diakui secara berurutan. `pending` menghitung pesan yang belum dikirim, dan `inflight` menghitung pesan yang sudah dikirim tetapi belum di-ACK. Karena dua jenis antrean ini berbeda, pending nol saja belum membuktikan recovery selesai.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 308,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 308,
      "pending": 0,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 308,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.1: Posisi awal stream dan dua durable utama sebelum Dashboard dihentikan.

Pada snapshot awal, Dashboard dan Notifier sama-sama memiliki ACK 308, sesuai sequence terakhir stream. Ini menunjukkan keduanya membaca stream yang sama dengan posisi masing-masing.

8.3.2 Menghentikan satu consumer dan membaca backlog

Dashboard dihentikan melalui Compose, sementara producer dan Notifier dibiarkan berjalan. Jeda 15 detik memberi kesempatan bagi mock untuk menghasilkan event baru dan Aggregator untuk meneruskannya ke broker.

```
$ docker compose stop dashboard-consumer
 Container aat-report-revision-dashboard-consumer-1  Stopping
 Container aat-report-revision-dashboard-consumer-1  Stopped

$ sleep 15
```

Gambar 8.2: Menghentikan Dashboard tanpa menghentikan producer, broker, atau Notifier.

Command monitoring yang sama kemudian dijalankan ulang. Sequence stream naik dari 308 menjadi 311 dan ACK Notifier ikut mencapai 311. ACK Dashboard tetap 308 dengan tiga pesan pending. Jadi, event baru tetap tersimpan dan diolah Notifier saat Dashboard tidak aktif. Sequence 311 dicatat sebagai target yang harus dicapai Dashboard setelah kembali berjalan.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                 pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 311,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 308,
      "pending": 3,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 311,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.3: Stream dan Notifier terus maju, sedangkan backlog Dashboard bertambah.

Dashboard dijalankan kembali, kemudian pengujian menunggu 35 detik sebelum membaca posisi ACK. Jeda ini juga memberi ruang untuk redelivery jika masih ada pesan yang dikirim sebelum consumer berhenti; konfigurasi AckWait adalah 30 detik.

```
$ docker compose start dashboard-consumer
 Container aat-report-revision-nats-1  Waiting
 Container aat-report-revision-nats-1  Healthy
 Container aat-report-revision-dashboard-consumer-1  Starting
 Container aat-report-revision-dashboard-consumer-1  Started

$ sleep 35
```

Gambar 8.4: Menjalankan kembali Dashboard dan menunggu pemrosesan backlog.

Pada pemeriksaan berikutnya, ACK Dashboard mencapai 318, melewati target 311 yang dicatat saat ia mati. Notifier juga mencapai 318, dan kedua consumer tidak mempunyai pesan pending maupun inflight pada snapshot tersebut. Hasil ini membuktikan Dashboard memproses event yang terbit selama gangguan, bukan hanya menerima event setelah startup.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | select(.name == "dashboard-v1" or .name == "notifier-v1")
>             | {name, ack: .ack_floor.stream_seq,
>                pending: .num_pending, inflight: .num_ack_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 318,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 318,
      "pending": 0,
      "inflight": 0
    },
    {
      "name": "notifier-v1",
      "ack": 318,
      "pending": 0,
      "inflight": 0
    }
  ]
}
```

Gambar 8.5: Dashboard sudah mengakui seluruh pesan hingga melewati target periode gangguan.

Inspeksi database lokal kemudian menunjukkan bahwa kedua consumer masing-masing memiliki 319 pesan pada inbox dan quarantine nol. Pemeriksaan ini dilakukan setelah snapshot broker, sehingga jumlah inbox dapat bertambah ketika event baru masuk. Inbox yang terpisah mendukung bahwa setiap aplikasi mengolah pesan secara independen.

```
$ docker compose exec -T dashboard-consumer /dashboard-consumer inspect \
>   | jq '{inbox_count, quarantine_count}'
{
  "inbox_count": 319,
  "quarantine_count": 0
}

$ docker compose exec -T notifier-consumer /notifier-consumer inspect \
>   | jq '{inbox_count, quarantine_count}'
{
  "inbox_count": 319,
  "quarantine_count": 0
}
```

Gambar 8.6: Hasil olahan berada pada inbox lokal masing-masing consumer.

8.3.3 Menambahkan consumer tanpa mengubah producer

Penambahan consumer diuji pada sesi terpisah. Sebelum Third Consumer dijalankan, ID, waktu mulai, dan restart count Aggregator disimpan. Profile `demo` kemudian menambahkan container Third Consumer dengan durable `third-consumer-v1`, tanpa membangun atau menjalankan ulang Aggregator. Consumer ini memakai binary Notifier dengan durable dan volume miliknya sendiri.

```
$ docker inspect $(docker compose ps -q aggregator) \
>   | jq -S '.[0] | {id: .Id, started_at: .State.StartedAt,
>       restarts: .RestartCount}' > /tmp/report-session/producer-before.json
> docker compose --profile demo up -d --no-deps --wait third-consumer
> sleep 5
#0 building with "default" instance using docker driver

#1 [third-consumer internal] load build definition from Dockerfile
```

```
#15 DONE 0.1s
 Volume "aat-report-revision_third-consumer-data"  Creating
 Volume "aat-report-revision_third-consumer-data"  Created
 Container aat-report-revision-third-consumer-1  Creating
 Container aat-report-revision-third-consumer-1  Created
 Container aat-report-revision-third-consumer-1  Starting
 Container aat-report-revision-third-consumer-1  Started
 Container aat-report-revision-third-consumer-1  Waiting
 Container aat-report-revision-third-consumer-1  Healthy
```

Gambar 8.7: Cuplikan awal dan akhir command startup Third Consumer melalui profil demo; bagian tengah log build tidak ditampilkan.

Pada sesi tersebut, snapshot monitoring menunjukkan durable baru telah mencapai ACK 192, sama dengan posisi stream dan kedua durable lainnya. Karena consumer baru memakai DeliverAll, pesan yang masih tersimpan sebelum ia dibuat juga tersedia untuk dibaca; producer tidak perlu mengirim ulang secara khusus kepadanya.

```
$ docker compose exec -T nats wget -qO- \
>   'http://127.0.0.1:8222/jsz?streams=true&consumers=true' \
>   | jq '.account_details[].stream_detail[]
>       | {name, last_seq: .state.last_seq,
>           consumers: [.consumer_detail[]
>             | {name, ack: .ack_floor.stream_seq,
>                 pending: .num_pending}]}'
{
  "name": "HAZARD_EVENTS",
  "last_seq": 192,
  "consumers": [
    {
      "name": "dashboard-v1",
      "ack": 192,
      "pending": 0
    },
    {
      "name": "notifier-v1",
      "ack": 192,
      "pending": 0
    },
    {
      "name": "third-consumer-v1",
      "ack": 192,
      "pending": 0
    }
  ]
}
```

Gambar 8.8: Durable baru membaca stream yang sama sampai posisi ACK yang ditampilkan broker.

Perbandingan kondisi Aggregator sebelum dan sesudah menghasilkan file yang identik. Inspeksi Third Consumer juga menunjukkan inbox 193 dengan quarantine nol. Jumlah inbox diambil setelah snapshot broker, ketika stream masih menerima event. Dengan demikian, penambahan consumer berhasil tanpa perubahan container producer, dan consumer baru berhasil menyimpan hasil pengolahannya.

```
$ docker inspect $(docker compose ps -q aggregator) \
>   | jq -S '.[0] | {id: .Id, started_at: .State.StartedAt,
>       restarts: .RestartCount}' > /tmp/report-session/producer-after.json
> diff -s /tmp/report-session/producer-before.json \
>   /tmp/report-session/producer-after.json
> docker compose --profile demo exec -T third-consumer \
>   /notifier-consumer inspect | jq '{inbox_count, quarantine_count}'
>
Files /tmp/report-session/producer-before.json and /tmp/report-
session/producer-after.json are identical
{
  "inbox_count": 193,
  "quarantine_count": 0
}
```

Gambar 8.9: Producer tidak berubah dan consumer tambahan memiliki inbox sendiri.

Kriteria	Bukti
P5.1	Publish relay ke subject broker pada kode dan kenaikan sequence ketika Dashboard mati.
P5.2	Posisi durable terpisah pada Gambar 8.1 dan inbox masing-masing pada Gambar 8.6.
P5.3	Sequence 311 sebagai target gangguan, lalu ACK Dashboard 318 pada Gambar 8.3 dan 8.5.
P5.4	Startup Third Consumer, posisi ACK, dan kondisi producer pada Gambar 8.7–8.9.
P5.5	Pilihan fan-out, at-least-once, dan transaksi inbox idempoten dijelaskan pada bab ini.

8.4. Batasan

Consumer dapat mengejar backlog selama event yang dibutuhkannya masih tersedia di stream. Durable mempertahankan posisi baca, tetapi tidak memperpanjang retention. Jika consumer berhenti melewati batas retention, event yang sudah kedaluwarsa tidak dapat diambil kembali dari broker.

Konfigurasi satu replica juga belum menyediakan failover antarmesin. Di sisi consumer, idempotensi bergantung pada riwayat inbox yang tersimpan di volume lokal. Jika volume tersebut dihapus, consumer kehilangan catatan pesan yang pernah diolah. Retention broker dan keberlangsungan volume karena itu menjadi bagian dari batas jaminan pengiriman pada milestone 1.

KESIMPULAN

Berdasarkan tes pada profil milestone 1 yang diuji, platform dapat menyatukan data BMKG dan PVMBG ke dalam HazardEvent, menerima tambahan atribut tanpa restart atau migrasi, serta membatasi data sesuai identitas client. Ketika PVMBG mengalami outage, API tetap menyediakan data terakhir dengan penanda stale, sementara data BMKG tetap dapat diperbarui. Pemulihan sumber kemudian diikuti polling tanpa menjalankan ulang jalur baca.

Pemisahan consumer juga terbukti pada pengujian fan-out. Dashboard dapat dihentikan atau dibangun ulang ketika producer dan Notifier tetap berjalan, lalu melanjutkan pembacaan event yang tersimpan di broker. Consumer baru dapat membaca stream yang sama tanpa perubahan producer.

Pada load test 5 Oktober 2026 dengan 60 worker dan delay PVMBG 3 detik, p95 BMKG sebesar 124,52 ms, throughput 980,26 respons sukses per detik, dan error selain 429 sebesar 0,034%. Sampel TCP menunjukkan sedikitnya 50 koneksi selama 89,00 detik. Hasil ini memenuhi ambang P2 pada profil beban yang diuji.

Ownership storage, kontrak event, outbox, dan consumer idempoten menjadi dasar untuk pengembangan milestone berikutnya. Jika jumlah instance ditambah, polling dan relay perlu dikoordinasikan, sedangkan state Auth serta consumer perlu dikelola agar sesuai dengan pola deployment tersebut. Pengujian kapasitas dan kegagalan juga harus diulang; hasil satu host pada milestone 1 belum membuktikan high availability pada beberapa node.

Prioritas lanjutan adalah menelusuri kegagalan perubahan schema pada volume besar, mengurangi biaya snapshot yang terus tumbuh, dan menetapkan pembersihan data lokal. Sinkronisasi waktu perlu dijaga untuk expiry token serta freshness. Sebelum deployment di luar demo lokal, sistem juga membutuhkan TLS, pemisahan credential broker, dan mekanisme rotasi key.

PEMBAGIAN TUGAS

Anggota	Perancangan	Implementasi	Penulisan laporan
M Hazim R Prajoda 13523009	Kontrak sumber, aturan pemetaan, Canonical Store	Mock BMKG/PVMBG, Aggregator, polling dan penyimpanan	P1 dan aspek flexible storage P4
Adhimas Aryo Bimo 13523052	Identitas client, scope, token, isolasi request	Auth/Token Service, Client API, ketahanan dan load test	P2, P3, serta pengukuran API
Refki Alfarizi 13523002	Event fan-out, durable consumer, jaringan dan integrasi	JetStream, consumer, orchestration, trial integrasi dan demo console	P5, container P4, penyusunan dan penggabungan laporan

Tabel 10.1: Kontribusi anggota pada perancangan, implementasi, dan penulisan laporan.

DAFTAR REFERENSI

1. IF4031. (2026). *Tugas Besar IF4031 — Arsitektur Aplikasi Terdistribusi, Milestone 1: Sistem Terdistribusi Dasar*. Dokumen spesifikasi mata kuliah.
2. IF4031. (2026). *Tugas Besar IF4031 — Arsitektur Aplikasi Terdistribusi*. Dokumen utama tugas besar.
3. Go Authors. *Package net/http*. `https://pkg.go.dev/net/http`.
4. Go Authors. *Package encoding/json*. `https://pkg.go.dev/encoding/json`.
5. PostgreSQL Global Development Group. *JSON Types*. `https://www.postgresql.org/docs/current/datatype-json.html`.
6. NATS.io. *JetStream*. `https://docs.nats.io/nats-concepts/jetstream`.
7. NATS.io. *JetStream Consumers*. `https://docs.nats.io/nats-concepts/jetstream/consumers`.
8. NATS.io. *JetStream Retention Policies*. `https://docs.nats.io/learn/jetstream/retention-policies`.
9. Docker. *Docker Compose documentation*. `https://docs.docker.com/compose/`.
10. Jones, M., Bradley, J., dan Sakimura, N. (2015). *JSON Web Token (JWT), RFC 7519*. `https://www.rfc-editor.org/rfc/rfc7519`.

FORMAT DEKLARASI PENGGUNAAN AI
SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya/~~kami*:

Nama/NIM	:	Refki Alfarizi / 13523002 M Hazim R Prajoda / 13523009 Adhimas Aryo Bimo / 13523052
Fakultas/Sekolah	:	Sekolah Teknik Elektro dan Informatika (STEI), ITB
Kode Mata Kuliah	:	IF4031
Nama Mata Kuliah	:	Arsitektur Aplikasi Terdistribusi
Judul Tugas/Proposal	:	Tugas Besar IF4031 - milestone 1: Sistem Terdistribusi Dasar

Menyatakan bahwa ~~saya/~~kami* menggunakan/~~tidak menggunakan*~~ AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

J yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Ya	Terbatas: penyuntingan bahasa laporan.
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Ya	Draf laporan dan kode Go; bantuan tes/debugging.
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Ya	Diskusi desain, analisis kode, dan verifikasi.
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Ya	Draf diagram Mermaid/TikZ; bukan foto/video.
Lainnya, Jelaskan:	Tidak	

~~Saya/~~Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 7 September 2026



Refki Alfarizi	M Hazim R Prajoda	Adhimas Aryo Bimo
13523002	13523009	13523052